Wear Before Capacity: Lifetime-Budgeted Prefix KV Caching on High Bandwidth Flash

H. R. J. Nikzat
Bates College, Lewiston, ME 04240 USA · rnikzat@bates.edu

Preprint. Submitted to IEEE Computer Architecture Letters on 24 October 2026; under review.

***Abstract*—Most High Bandwidth Flash (HBF) studies of the key-value (KV) cache of large language models assume single-level cells rated for 100,000 program/erase cycles. We ask what a five-year life allows a prefix KV cache on denser cells, on Mooncake and Qwen-Bailian production traces. At its life budget a region keeps each block for a time that does not depend on its capacity, so writable bytes decide what the cache can do, and cell mode moves GPU-time savings as much as the admission policy does. On 8 stacks, three- and four-bit cells can write only 1.7–3.9% of the bytes HBM evicts for Llama-3.1-70B and 8–18% for DeepSeek-V3's latent KV. We then price wear: a block is written only when its expected recomputation savings exceed the price of its wear, and wear stays provably at most a constant ahead of the life schedule. At a 10% budget the price keeps 35.4% and 32.1% of the GPU time that writing every block saves, 5.0–6.0 points more than a lifetime quota given the load curve, at equal bytes. Its gain comes from pacing writes to the load: without a load curve it keeps 0.3–1.2 points more than a budget-tracking frequency rule given one. A NAND-write counter keeps wear on budget under unknown write amplification.**

***Index Terms*—Admission control, caches, endurance, flash memory, KV cache, large language models.**

I. Introduction

HIGH Bandwidth Flash (HBF) stacks NAND dies like high bandwidth memory (HBM) and connects them to an accelerator, trading write endurance and latency for capacity [1]. Studies from 2026 place model weights and key-value (KV) data in HBF, and most estimate its life assuming single-level cells (SLC) rated for 100,000 program/erase (P/E) cycles [2]–[7]. Denser cells cost less per byte and endure fewer cycles: 1,000–3,000 for three-bit cells (TLC) [8], [9] and about 1,000 for four-bit cells (QLC) under sequential writes [10].

The HBF specification leaves endurance to the host: the host is responsible for determining a device's remaining lifespan from its average and maximum P/E counts (Sec. 9) and may run wear leveling itself [1]; what to write is up to the host. This letter supplies that host policy for a prefix KV cache, which keeps blocks evicted from HBM for later requests [11]. We contribute (1) an endurance analysis: at a life budget, residence time does not depend on capacity, so writable bytes decide what the cache can do, far more for grouped-query attention (Llama-3.1-70B) than for multi-head latent attention (MLA, DeepSeek-V3); (2) ROAR (read-often, append-rarely), an admission rule that spends the rated life, stays at most a proven constant ahead of the service-life schedule and needs no model of the daily load; and (3) a comparison with lifetime quotas, frequency rules and an offline oracle.

II. Background and Related Work

HBF and KV cache. Prior HBF letters size or place KV so that SLC lasts. Baek *et al.* write paused sessions' KV once, on eviction, to 3 TiB of SLC and find it lasts about 20 years with 48 sessions decoding at once [7]; with their WAF of 1.02 that implies about 44 TB of writes per day, 8.6% of what the same rule (our write-all) writes on our tool-and-agent day, at which their SLC would last 1.7 years and TLC 19 days. Peng *et al.*'s LRU-K admission filter stretches HBF life from about one year to over six [12]; ROAR instead sets the write rate from the life. These designs lower the write rate, but life still depends on load: 2.0–6.9 years of SLC life in HBFSim [6], at most 2.3 years once read reclaim is counted [13]. For SSD prompt caches, Lee *et al.* size capacity so that the daily write volume stays within SLC, TLC or QLC drive-write ratings [14]; we fix the device instead and choose which blocks it can afford to write. To our knowledge, no prior HBF design sets its KV write rate from the rated life of dense cells.

Flash caches. READY and two patents set flash write budgets from the rated life, by throttling or by per-period quotas [15]–[17]. Baleen learns admission under a fixed flash write rate [18]; ROAR derives the rate from the rated life and spends it unevenly over the day. These throttles and quotas follow a fixed schedule and need a load model to spend the budget at the right hours; a price does not. Our price update is standard dual ascent on a long-term constraint [19].

III. Wear Binds Before Capacity

A region of capacity C with rated cycles N and write amplification factor WAF may absorb $B = CN/(D \cdot \mathrm{WAF})$ host bytes per day over a life of D days. If a policy writes at exactly B into a first-in, first-out (FIFO) log, each block stays resident for

$$R = \frac{C}{B} = \frac{D \cdot \mathrm{WAF}}{N}, \qquad (1)$$

independent of C. In days, R is the reciprocal of the drive-writes-per-day rating that Lee *et al.* use to size SSD prompt caches [14]. What is new is the consequence: endurance alone sets residence, so cell mode decides which reuse gaps a cache can serve, and for reuse within R the figure of merit is writable bytes, $C{\cdot}N$. With WAF = 1, which HBFSim measures for KV appended and freed in bulk [6], R is 26 minutes for SLC (100K cycles), 2.2 hours for pseudo-SLC (pSLC; 20K, a vendor's rating for 3D TLC dies [9]), 14.6 hours for TLC (3K) and 43.8 hours for QLC (1K) at a five-year life, the warranty Lachesis assumes [5] (a ten-year target halves every budget). We use 3K for TLC, the upper end of its ratings; at 1K its

budget falls below QLC's and pSLC breaks even at 3K. A WAF of 2 halves every budget and doubles every R. Behind a 1 TB HBM cache (Section V), the median time from a block's HBM eviction to its next reference within the one-hour trace is 108 s (tool and agent) and 132 s (conversation). At SLC endurance, writing every evicted block (write-all) lasts 2.2 years with 8 stacks per node; at TLC or QLC it lasts weeks to months, and the budget admits 1.7–16% of evicted bytes for Llama-3.1-70B (Table I). DeepSeek-V3's MLA stores 4.66× less KV per token; write-all writes each block at most once whatever the HBM size, so it also writes 4.66× fewer bytes, and 8 TLC stacks allow 18% (QLC: 8%). Dense HBF is wear-starved for grouped-query attention but usable for MLA.

Running TLC dies as pSLC stores one bit per cell instead of three, so it keeps a third of the capacity and, for equal die cost and equal WAF, gains writable bytes only if pSLC endures more than 3× the TLC rating, 9K cycles. As a design rule, 8 pSLC stacks allow an 8.7% budget and 32 TLC stacks 15.7%, with 12× the capacity and 6.7× the residence, which helps only for reuse between the two.

TABLE I
FIVE-YEAR WRITE BUDGET PER NODE

Cell (cycles)	Budget share		Write-all life (y)		Residence
	8 st.	32 st.	8 st.	32 st.	at budget
SLC (100K)	43.6%	174.6%	2.18	8.73	26 min
pSLC (20K)	8.7%	34.9%	0.44	1.75	2.2 h
TLC (3K)	3.9%	15.7%	0.20	0.79	14.6 h
QLC (1K)	1.7%	7.0%	0.09	0.35	43.8 h

Share of write-all bytes (Llama-3.1-70B, tool-and-agent day: 514 TB), years until write-all wears the device out, FIFO residence. Stacks (assumed; the specification's cube is 512 GiB, cell type unspecified [1]): SLC, pSLC 512 GB; TLC 1.536 TB; QLC 2.048 TB.

IV. WEAR-PRICED ADMISSION

When HBM evicts block u that is not in flash, ROAR values a stored copy as

$$v_u = \hat{h}_u\,(t_{\text{rec}}(d_u) - t_{\text{read}}), \qquad t_{\text{rec}}(d) = \frac{512\,(F_{\text{lin}} + F_{\text{att}}\,d)}{\Phi}, \qquad (2)$$

where $\hat{h}_u$ is the expected number of later references that will miss HBM (in its simplest form, the block's reference count so far), d_u the block's depth in tokens, 512 the block length in tokens, F_{lin} the dense-layer FLOPs per token (141.2 GFLOP for Llama-3.1-70B; about 3% high, since it counts the embedding and output layers), F_{att} the attention FLOPs per token per earlier token (2.62 MFLOP), Φ the GPU throughput (1.12 PFLOP/s: a B200 at 50% of its 2.25 PFLOP/s dense BF16 peak), and t_{read} the time to read the 160 MiB block from four stacks at 1.6 TB/s each (near the specification's 1.536 TB/s speed grade [1]; 26 μs), far below $t_{\text{rec}}(0) =$ 64 ms. Writing u to region r costs $c_{u,r} = s_u/b_r$: the block's size s_u over b_r, the region's byte budget for one control interval. The block goes to $r^* = \text{argmax}_r\big(v_u - \lambda_r c_{u,r}\big)$ if that is positive; otherwise it is dropped. A block whose parent is in neither HBM nor flash is not written, since a prefix lookup could not reach it.

Learned estimate (optional). The results use a refinement worth under a point at equal bytes: $\hat{h}_u$ is the mean count of HBM-missing references in the 15 minutes after past evictions of the same class (reference count, capped at 6; whether a request has continued past the block; position), shrunk toward coarser classes.

Price. After each 10-second interval the controller sets $\lambda_r \leftarrow \max\big(\lambda_{\min}, \lambda_r e^{\eta(w_r - 1)}\big)$, where w_r is the interval's wear in units of the interval budget; the schedule spends the life budget evenly, and the lead is cumulative wear minus the schedule. The controller counts host bytes. N is the maximum P/E count the device reports [1]. For WAF and internal writes such as read reclaim [13], the controller reads the device's count of bytes written to NAND, which datacenter SSDs report [20]; every hour it sets the host budget to the NAND budget over the ratio of NAND to host bytes in the last six hours. It needs no device model. Let $\rho_{\max}$ bound v/c and $w_{\max}$ bound one interval's wear. For every horizon T and starting price $\lambda_r(1) \le \rho_{\max}$,

$$\sum_{t \le T} w_r\,(t) \le T + \eta^{-1}\ln\big(\rho_{\max}/\lambda_r(1)\big) + w_{\max} - 1. \qquad (3)$$

Proof sketch (dropping r). The floor gives $\lambda(T+1) \ge \lambda(1)\exp(\eta\sum_t (w(t) - 1))$. If $\lambda \ge \rho_{\max}$ nothing is written and the price falls, so λ never exceeds $\rho_{\max} e^{\eta(w_{\max} - 1)}$; taking logarithms gives the bound [19]. A slow price lets the lead grow, still bounded, and moves writes toward the hours when blocks are worth most. We use $\eta = 0.05$ for the first six hours after installation and $\eta = 0.0003$ afterwards, both chosen on the FAST'25 synthetic trace [11], which shares no requests with the evaluation traces; (3) holds for each phase. The price starts where a block read once at depth 0 is worth its wear ($\lambda(1) =$ 2.29 GPU-seconds per interval budget), $\lambda_{\min} = 10^{-6}\lambda(1)$, and the largest v/c and interval wear seen were $\rho_{\max} = 7.86$ and $w_{\max} = 16.29$. With the price at the switch, 0.286, the two phases' bounds sum to 1.3 days of budget (conversation, 10%).

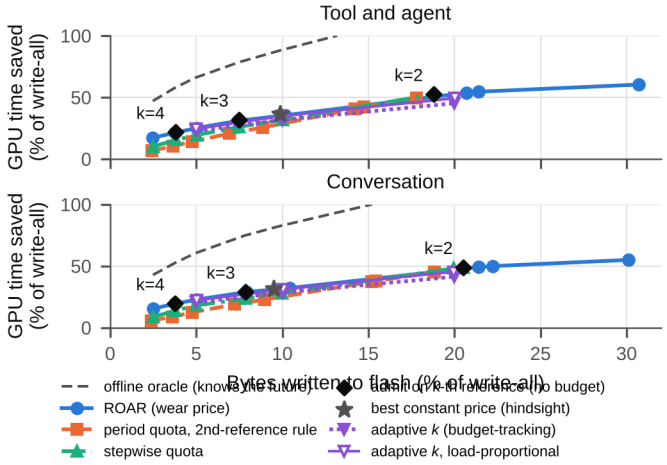


Fig. 1. GPU time saved by flash hits against flash bytes written on day 2 (known load, seed pair 0–1), both as shares of write-all. Lines: budgets of 2.5–30%; diamonds: fixed rules with no budget; star: best constant price in hindsight, 10%.

V. Methodology

We use the two real workloads of the Mooncake FAST'25 release [11]: tool-and-agent (23,608 requests) and conversation (12,031 requests), each one hour, with 512-token blocks named by prefix hashes. Flash stores every block padded to 512 tokens (160 MiB), as paged KV caches allocate whole blocks. A least-recently-used (LRU) HBM cache of 6,000 blocks (1.0 TB of Llama-3.1-70B KV) sits in front of flash. Every referenced block enters HBM, so HBM evictions do not depend on the flash policy; a request uses cached blocks up to its first miss. A synthetic day keeps each request of the hour with a probability that follows a daily cosine from 0.3 to 1.0, peaking at 14:00, and gives each hour its own block names. Each run covers two days and is measured on the second, with the budget a fixed share of that day's write-all bytes spread evenly over time; we use three seed pairs per trace. On *unknown-load* days (the second load family), the trough (0.2–0.5), the peak hour (11:00–17:00) and a one-hour burst at 1.5× load are drawn anew each day. The budget is then fixed bytes per day, a share of the write-all bytes of a day with the family's mean curve. A node has 8 stacks (one per GPU) or 32. In the policy comparison flash capacity is unlimited: at its life budget a pSLC, TLC or QLC node keeps blocks at least 2.2 hours, and a synthetic day's hours share no blocks. The cell-mode and long-gap runs use FIFO logs of the node's capacity.

Adaptive k admits a block on its *k*-th reference and moves a real-valued *k* by 0.01 every 10 seconds as wear lags or leads the schedule, rounding it at random per block; of 11 variants it kept the most at 10% on both real traces. The *period quota* admits second-reference blocks in arrival order up to the remaining budget over the remaining minutes of a five-year life [16]. The *stepwise quota* admits more strictly as each quantum's allowance is used up [17], with the setting chosen on the synthetic trace (300-second quanta, steps of 2/3/4 references, and 2 references even within the allowance). Adaptive *k* and the period quota also run *load-proportional*, with a schedule that follows the expected load curve: the true one on known days, the family's mean (trough 0.35, peak 14:00) on unknown days. A *constant price* is set to spend exactly the budget on day 1 and held on day 2. All baselines are our reimplementations. An offline oracle knows each block's future HBM misses and selects greedily, adding missing ancestors.

VI. Results

TABLE II
Day-2 GPU Time Saved by Flash Hits on Known Load

	Tool and agent			Conversation		
Budget	5%	10%	20%	5%	10%	20%
ROAR (this work)	25.0	35.4	54.3	22.8	32.1	49.0
value = ref. count	24.3	34.6	52.7	22.1	31.0	48.7
Adaptive *k*	22.6	31.6	45.0	20.4	28.7	41.1
load-prop.	24.4	34.7	49.2	22.0	31.3	45.6
Period quota	14.1	25.7	42.2	12.1	22.4	36.9
load-prop.	16.3	29.5	47.7	14.1	25.5	43.0
Stepwise quota	19.4	31.8	52.2	17.6	27.8	47.9
Constant price, day 1	28.9	34.7	58.2	24.6	30.9	51.4
Offline oracle*	66.1	88.4	100.0	60.5	82.7	100.0

Percent of write-all, mean of three seed pairs (spread ≤1.6 points). *Knows the future; spends the day's bytes.

TABLE III
ROAR's Lead at Equal Bytes, 10% Budget

	Known load			Unknown load		
	ROAR lead			ROAR lead		
Policy	TA	CV	W/B	TA	CV	W/B
ROAR (this work)	–	–	1.03	–	–	1.04
value = ref. count	+0.5	+0.7	1.00	+0.7	+0.6	1.02
Adaptive *k*	+3.5	+3.0	1.00	+3.8	+3.0	1.00
load-prop.	+0.4	+0.3	1.00	+1.2	+0.9	1.00
yesterday's load	+0.4	+0.4	1.00	+2.0	+1.2	1.00
Period quota	+7.8	+8.0	0.89	+7.8	+7.5	0.91
load-prop.	+5.0	+5.8	0.96	+6.0	+5.8	0.96
Stepwise quota	+3.3	+3.9	1.00	+3.3	+3.7	1.02
Constant price, day 1	−0.9	−1.0	0.89	−0.9	−0.9	1.02

Mean of three seed pairs (load-aware adaptive *k* on unknown load: 9 pairs). Lead: points of savings over each policy at its bytes (ROAR's sweep, interpolated). W/B: day-2 writes per budget. TA: tool and agent; CV: conversation; – : reference row.

Cell mode against policy. Writing every evicted block would save 16.2 (tool and agent) and 18.4 (conversation) GPU-hours per day, 12% and 18% of the GPU time to prefill every prompt. On 8 TLC stacks per node, ROAR saves 3.5 GPU-hours per day on tool and agent and the period quota 1.8; the same dies run as pSLC let ROAR save 5.6 (it wrote 1.06× its budget). Cell mode thus moves savings at least as much as the policy does. On a fixed node the life target sets the budget; written exactly at it, 8 pSLC stacks let ROAR save 3.7, 5.4 and 7.1 GPU-hours per day for 10-, 5- and 3-year lives. Below we use budgets of 5–20%, spanning 8 pSLC and 32 TLC stacks (Table I); our claims are for 5–10%, where the budget binds hardest.

Known load. At the bytes each policy wrote, ROAR keeps 0.4 and 0.3 points more than adaptive *k* given the true load curve, and 5.0 and 5.8 more than the load-proportional quota, at a 10% budget (Table III, Fig. 1). Over both traces and load families, the lead over load-proportional adaptive *k* is 0.5–0.7 points at 5% and 0.9–3.3 at 20%. In a 5-day run measured on days 3–5 (conversation), ROAR kept 31.9% at 1.00× budget with wear at most 0.59 days of budget ahead of schedule, within (3); adaptive *k* given the load curve kept 31.0% and the period quota 22.5%.

Unknown load. With a fixed byte budget and over 9 seed pairs per trace, ROAR's lead at equal bytes over adaptive *k* given the mean curve was 1.2 ± 0.3 and 0.9 ± 0.2 points (95%

confidence), and 2.0 ± 1.0 and 1.2 ± 0.7 over adaptive k that follows the previous day's measured load.

Second source. On 8 Qwen-Bailian hours [21] (chat, coding, reasoning and API traffic), ROAR at 10% led the quotas by 0.0–13.7 points at equal bytes and adaptive k given the true load curve by −0.8 to 20.4. Adding each request's turn number to the estimator's classes raised ROAR's savings by 0.5–12.0 points, 1–20% of the gap to the oracle.

Two rules for any policy. Skipping each prompt's partial last block, which only an identical prompt can reuse, removes 11.9% and 6.0% of write-all's bytes and 0.2% and 0.1% of its savings; ROAR's estimate already avoids such blocks. The parent check matters more: without it ROAR kept 24.5% instead of 35.4%, and adaptive k 27.1% instead of 31.7% (tool and agent, 10%).

Unknown WAF. With a true WAF of 1.5, 2, a step from 1.5 to 2, or hourly values in 1.3–2.2, reading the NAND counter kept day-2 NAND writes at 0.98–1.03 (tool and agent) and 1.01–1.04 (conversation) of budget. Savings stayed within 1.0 points of a controller told the true WAF; assuming $\mathrm{WAF} = 1$ wrote 1.49–2.14×.

The value model is optional. With the reference count in place of the reuse estimate, ROAR keeps 0.8 and 1.1 points less at 10% (0.5–0.7 at equal bytes). A constant price set on day 1 kept 0.9–1.0 points more at equal bytes but wrote 0.84–1.30× its budget: a price must follow the load.

Long gaps. In 179,571 WildChat-1M conversations [22], 4.4% of follow-up turns came over an hour after the previous reply. Re-issuing requests at that rate and with those gaps (conversation, 8 stacks), pSLC still beat TLC (ROAR: 5.5 against 4.0 GPU-hours per day); TLC won only when 20% of requests returned after 1–24 hours.

VII. Conclusion and Limitations

For reuse within an hour, a prefix KV cache on TLC or QLC HBF is limited by writable bytes, not capacity, for grouped-query attention; MLA's smaller KV leaves room for TLC. Returns as rare as WildChat's leave pSLC ahead of TLC. A slow wear price stays at most a proven constant ahead of the schedule and, without a load curve, keeps slightly more than the best budget rule given one; its gain is timing, since at equal bytes a constant price or a frequency rule selects about as well. Any policy should write only blocks a prefix lookup can reach; skipping partial last blocks saves 6.0–11.9% of write-all's bytes almost for free. Limits: each trace covers one or two hours and the daily load is synthetic; reuse beyond an hour is probed only by applying WildChat's turn gaps to Mooncake requests, and the seed pairs re-sample one hour rather than independent workloads. We model no queueing, write-read interference, ECC, retention loss or read reclaim (no block was read more than 18 times); WAF enters only as a scaling of wear, and recomputation time comes from FLOPs. The adaptive-k setting was picked on the evaluation traces, which favors that baseline. Code: github.com/ho3nik/SHIR-ROAR.

Acknowledgment

The author thanks Dr. Abel Beregi, Postdoctoral Researcher at Bates College, for providing the IEEE papers referenced in this letter, and his brother & raison d'être, Soheil Sadabadi (CTO, Praxos), who read the manuscript closely and offered constructive criticism.

References

[1] C. Ballapuram and D. Lee, "High bandwidth flash (HBF) high-level base die specification," Open Compute Project, 2026, Version 0.7.0, 3 Aug. 2026. [Online]. Available: https://www.opencompute.org/documents/ocp-hbf-architecture-specification-v0-7-0-final-pdf/ (accessed 23 Oct. 2026).

[2] D. Son *et al.*, "Exploring high-bandwidth flash for modern LLM inference: Opportunities and challenges," *IEEE Computer Architecture Letters*, vol. 25, no. 2, pp. 251–254, 2026.

[3] K. Kyung, Y. J. Moon, J. Cho, and J. H. Ahn, "High-bandwidth flash for KV caches: Endurance and performance implications," *IEEE Computer Architecture Letters*, vol. 25, no. 1, pp. 210–213, 2026.

[4] Z. Yu *et al.*, "Characterizing high bandwidth flash for LLM serving," 2026, arXiv:2609.39131.

[5] J. Yang, J. Lee, H. Park, S. Y. Lee, N. S. Kim, and J. W. Lee, "Lachesis: Lifetime-aware KV cache placement for agent serving across HBM and high-bandwidth flash," 2026, arXiv:2610.08378.

[6] J. Li, G. Wang, H. Du, S. Wu, Q. Li, and C. J. Xue, "HBFSim: A simulator for exploring HBF in memory hierarchies for LLMs," in *Proc. RACS*, 2026, arXiv:2610.23503.

[7] J. Baek, W. Ji, S. Yoo, and J.-Y. Kim, "Hot–cold tiering of HBM and high bandwidth flash for agentic LLM serving," *IEEE Computer Architecture Letters*, vol. 25, no. 2, pp. 355–358, 2026.

[8] B. Kim, G. Seo, and M. Kim, "Smart electrical screening methodology for channel hole defects of 3D vertical NAND (VNAND) flash memory," *Eng*, vol. 5, no. 1, pp. 495–512, 2024.

[9] Virtium, "How industrial SSDs may use 3D TLC NAND flash to create SLC-like endurance," 2021, technology update, 29 Jul. 2021.

[10] A. Tai, A. Kryczka, S. O. Kanaujia, K. Jamieson, M. J. Freedman, and A. Cidon, "Who's afraid of uncorrectable bit errors? Online recovery of flash errors with distributed redundancy," in *Proc. USENIX ATC*, 2019.

[11] R. Qin *et al.*, "Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot," in *Proc. USENIX FAST*, 2025.

[12] D. Peng, K. Wu, T. Zuo, P. Xia, and H. Zang, "Enabling high-bandwidth flash for generative recommendation serving with write-aware KV cache policy," 2026, arXiv:2609.07175.

[13] I. Choi, H. Kim, J. Lee, and J.-J. Kim, "Read-reclaim-induced endurance degradation in high-bandwidth flash for LLM inference," *IEEE Computer Architecture Letters*, 2026, early access, doi: 10.1109/LCA.2026.3742410.

[14] J. Lee, K. Ko, T. Hwang, W. Chung, H. Kim, and D. Moon, "Prompt caching on flash: Achieving memory-class latency for LLM inference," *IEEE Computer Architecture Letters*, vol. 25, no. 2, pp. 295–298, 2026.

[15] S. Lee, T. Kim, K. Kim, and J. Kim, "Lifetime management of flash-based SSDs using recovery-aware dynamic throttling," in *Proc. USENIX FAST*, 2012.

[16] Nimble Storage, Inc., "Methods and systems for reducing churn in flash-based cache," 2015, U.S. Patent 9,213,628 B2.

[17] EMC IP Holding Co., "Flash cache throttling to control erasures," 2020, U.S. Patent 10,877,879 B1.

[18] D. L.-K. Wong *et al.*, "Baleen: ML admission & prefetching for flash caches," in *Proc. USENIX FAST*, 2024.

[19] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool, 2010.

[20] Open Compute Project, "Datacenter NVMe SSD specification," 2023, Version 2.5, requirement SMART-1.

[21] J. Wang, J. Han, X. Wei, S. Shen, D. Zhang, C. Fang, R. Chen, W. Yu, and H. Chen, "KVCache cache in the wild: Characterizing and optimizing KVCache cache at a large cloud provider," in *Proc. USENIX Annu. Tech. Conf. (ATC)*, 2025, pp. 465–482, traces: github.com/alibaba-edu/qwen-bailian-usagetraces-anon.

[22] W. Zhao, X. Ren, J. Hessel, C. Cardie, Y. Choi, and Y. Deng, “WildChat: 1M ChatGPT interaction logs in the wild,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.